Day 4

평가 · 운영 · 확장 아키텍처 전략

AI Agent 전문 개발 과정

Session 1

Agent 품질 평가 체계 설계

품질 평가 없이 일어나는 일

품질 평가 없이 일어나는 일

- "잘 되는 것 같다"는 직관만 남는다

품질 평가 없이 일어나는 일

- "잘 되는 것 같다"는 직관만 남는다
- 버그가 생겨도 언제부터인지 알 수 없다

품질 평가 없이 일어나는 일

- "잘 되는 것 같다"는 직관만 남는다
- 버그가 생겨도 언제부터인지 알 수 없다
- 개선 방향을 숫자 없이 결정한다

품질 평가 없이 일어나는 일

- "잘 되는 것 같다"는 직관만 남는다
- 버그가 생겨도 언제부터인지 알 수 없다
- 개선 방향을 숫자 없이 결정한다
- 신뢰할 수 없는 시스템이 된다

품질 평가 없이 일어나는 일

- "잘 되는 것 같다"는 직관만 남는다
- 버그가 생겨도 언제부터인지 알 수 없다
- 개선 방향을 숫자 없이 결정한다
- 신뢰할 수 없는 시스템이 된다

> 금융 기업 사례: Faithfulness 0.6 → 원인 불명 → 평가 도입 후 청크 크기 조정으로 0.85 달성

품질 평가 3축

Accuracy

정답과 얼마나 일치하는가

최종 응답 vs. 기대 응답

Faithfulness

출처에 근거한 응답인가

응답 vs. 검색된 문서

Robustness

입력 변형에 일관된가

표현 바꿔도 같은 답?

평가 레벨 구조

```
Unit Level    → 단일 LLM 호출 품질
Step Level    → 하나의 Agent 스텝 (Tool 호출 포함)
Task Level    → 전체 Task 완료 여부
System Level  → 비용 · 안전성 · UX
```

평가 레벨 구조

```
Unit Level    → 단일 LLM 호출 품질
Step Level    → 하나의 Agent 스텝 (Tool 호출 포함)
Task Level    → 전체 Task 완료 여부
System Level  → 비용 · 안전성 · UX
```

- **Unit/Step** → CI/CD에서 자동화

평가 레벨 구조

```
Unit Level    → 단일 LLM 호출 품질
Step Level    → 하나의 Agent 스텝 (Tool 호출 포함)
Task Level    → 전체 Task 완료 여부
System Level  → 비용 · 안전성 · UX
```

- **Unit/Step** → CI/CD에서 자동화
- **Task** → 스테이징 Golden Test Set 통과 기준

평가 레벨 구조

```
Unit Level    → 단일 LLM 호출 품질
Step Level    → 하나의 Agent 스텝 (Tool 호출 포함)
Task Level    → 전체 Task 완료 여부
System Level  → 비용 · 안전성 · UX
```

- **Unit/Step** → CI/CD에서 자동화
- **Task** → 스테이징 Golden Test Set 통과 기준
- **System** → 주간/월간 리뷰

정량 vs. 정성 평가

정량 평가

정성 평가

정량 vs. 정성 평가

정량 평가

- 자동화 가능 · 빠른 피드백

정성 평가

정량 vs. 정성 평가

정량 평가

- 자동화 가능 · 빠른 피드백
- ROUGE, BLEU, EM, F1

정성 평가

정량 vs. 정성 평가

정량 평가

- 자동화 가능 · 빠른 피드백
- ROUGE, BLEU, EM, F1
- 재현 가능

정성 평가

정량 vs. 정성 평가

정량 평가

- 자동화 가능 · 빠른 피드백
- ROUGE, BLEU, EM, F1
- 재현 가능
- 의미적 정확성을 놓칠 수 있음

정성 평가

정량 vs. 정성 평가

정량 평가

- 자동화 가능 · 빠른 피드백
- ROUGE, BLEU, EM, F1
- 재현 가능
- 의미적 정확성을 놓칠 수 있음

정성 평가

- 사람이 직접 판단

정량 vs. 정성 평가

정량 평가

- 자동화 가능 · 빠른 피드백
- ROUGE, BLEU, EM, F1
- 재현 가능
- 의미적 정확성을 놓칠 수 있음

정성 평가

- 사람이 직접 판단
- 높은 신뢰도

정량 vs. 정성 평가

정량 평가

- 자동화 가능 · 빠른 피드백
- ROUGE, BLEU, EM, F1
- 재현 가능
- 의미적 정확성을 놓칠 수 있음

정성 평가

- 사람이 직접 판단
- 높은 신뢰도
- LM-as-a-Judge로 부분 대체

정량 vs. 정성 평가

정량 평가

- 자동화 가능 · 빠른 피드백
- ROUGE, BLEU, EM, F1
- 재현 가능
- 의미적 정확성을 놓칠 수 있음

정성 평가

- 사람이 직접 판단
- 높은 신뢰도
- LM-as-a-Judge로 부분 대체
- 비용·시간 소모 큼

LM-as-a-Judge

LM-as-a-Judge

- GPT-4/Claude 급 모델이 **평가자 역할**

LM-as-a-Judge

- GPT-4/Claude 급 모델이 **평가자 역할**
- 사람 판단과 상관관계 **0.8 이상**

LM-as-a-Judge

- GPT-4/Claude 급 모델이 **평가자 역할**
- 사람 판단과 상관관계 **0.8 이상**
- 비용은 사람보다 **10~100배 저렴**

LM-as-a-Judge

- GPT-4/Claude 급 모델이 **평가자 역할**
- 사람 판단과 상관관계 **0.8 이상**
- 비용은 사람보다 **10~100배 저렴**
- 서술형·다단계 평가에 강점

LM-as-a-Judge 편향 제거

```python
def pairwise_judge(question, response_a, response_b):
    # 순서 무작위화로 위치 편향 제거
    result1 = single_eval(response_a, response_b, "A", "B")
    result2 = single_eval(response_b, response_a, "B", "A")
    # 두 판정 일치 여부로 신뢰도 판단
    if result1["winner"] == "A" and result2["winner"] == "B":
        return {"winner": "A", "confidence": "high"}
    return {"winner": "tie", "confidence": "low"}
```

LM-as-a-Judge 편향 제거

- **위치 편향** → 응답 순서를 무작위로 바꿔 두 번 평가

```
def pairwise_judge(question, response_a, response_b):
    # 순서 무작위화로 위치 편향 제거
    result1 = single_eval(response_a, response_b, "A", "B")
    result2 = single_eval(response_b, response_a, "B", "A")
    # 두 판정 일치 여부로 신뢰도 판단
    if result1["winner"] == "A" and result2["winner"] == "B":
        return {"winner": "A", "confidence": "high"}
    return {"winner": "tie", "confidence": "low"}
```

LM-as-a-Judge 편향 제거

- **위치 편향** → 응답 순서를 무작위로 바꿔 두 번 평가
- **자기 선호 편향** → 평가 모델 ≠ 생성 모델

```
def pairwise_judge(question, response_a, response_b):
    # 순서 무작위화로 위치 편향 제거
    result1 = single_eval(response_a, response_b, "A", "B")
    result2 = single_eval(response_b, response_a, "B", "A")
    # 두 판정 일치 여부로 신뢰도 판단
    if result1["winner"] == "A" and result2["winner"] == "B":
        return {"winner": "A", "confidence": "high"}
    return {"winner": "tie", "confidence": "low"}
```

LM-as-a-Judge 편향 제거

- **위치 편향** → 응답 순서를 무작위로 바꿔 두 번 평가
- **자기 선호 편향** → 평가 모델 ≠ 생성 모델
- **보상 해킹** → 평가 기준 주기적 변경 + 사람 교차 검증

```
def pairwise_judge(question, response_a, response_b):
    # 순서 무작위화로 위치 편향 제거
    result1 = single_eval(response_a, response_b, "A", "B")
    result2 = single_eval(response_b, response_a, "B", "A")
    # 두 판정 일치 여부로 신뢰도 판단
    if result1["winner"] == "A" and result2["winner"] == "B":
        return {"winner": "A", "confidence": "high"}
    return {"winner": "tie", "confidence": "low"}
```

LM-as-a-Judge 편향 제거

- **위치 편향** → 응답 순서를 무작위로 바꿔 두 번 평가
- **자기 선호 편향** → 평가 모델 ≠ 생성 모델
- **보상 해킹** → 평가 기준 주기적 변경 + 사람 교차 검증

```
def pairwise_judge(question, response_a, response_b):
    # 순서 무작위화로 위치 편향 제거
    result1 = single_eval(response_a, response_b, "A", "B")
    result2 = single_eval(response_b, response_a, "B", "A")
    # 두 판정 일치 여부로 신뢰도 판단
    if result1["winner"] == "A" and result2["winner"] == "B":
        return {"winner": "A", "confidence": "high"}
    return {"winner": "tie", "confidence": "low"}
```

LM-as-a-Judge 편향 제거

- **위치 편향** → 응답 순서를 무작위로 바꿔 두 번 평가
- **자기 선호 편향** → 평가 모델 ≠ 생성 모델
- **보상 해킹** → 평가 기준 주기적 변경 + 사람 교차 검증

```
def pairwise_judge(question, response_a, response_b):
    # 순서 무작위화로 위치 편향 제거
    result1 = single_eval(response_a, response_b, "A", "B")
    result2 = single_eval(response_b, response_a, "B", "A")
    # 두 판정 일치 여부로 신뢰도 판단
    if result1["winner"] == "A" and result2["winner"] == "B":
        return {"winner": "A", "confidence": "high"}
    return {"winner": "tie", "confidence": "low"}
```

Golden Test Set 설계

```python
@dataclass
class GoldenTestCase:
    id: str
    category: str   # "accuracy" | "faithfulness" | "robustness"
    input: dict
    expected_output: str
    tolerance: float = 0.8  # 합격 기준 점수
```

Golden Test Set 설계

- **최소 50건**, 이상적으로 200~500건

```python
@dataclass
class GoldenTestCase:
    id: str
    category: str   # "accuracy" | "faithfulness" | "robustness"
    input: dict
    expected_output: str
    tolerance: float = 0.8  # 합격 기준 점수
```

Golden Test Set 설계

- **최소 50건**, 이상적으로 200~500건
- 카테고리별 균형 (Accuracy / Faithfulness / Robustness)

```python
@dataclass
class GoldenTestCase:
    id: str
    category: str   # "accuracy" | "faithfulness" | "robustness"
    input: dict
    expected_output: str
    tolerance: float = 0.8  # 합격 기준 점수
```

Golden Test Set 설계

- **최소 50건**, 이상적으로 200~500건
- 카테고리별 균형 (Accuracy / Faithfulness / Robustness)
- **격리된 저장소**에서 버전 관리

```python
@dataclass
class GoldenTestCase:
    id: str
    category: str   # "accuracy" | "faithfulness" | "robustness"
    input: dict
    expected_output: str
    tolerance: float = 0.8  # 합격 기준 점수
```

Golden Test Set 설계

- **최소 50건**, 이상적으로 200~500건
- 카테고리별 균형 (Accuracy / Faithfulness / Robustness)
- **격리된 저장소**에서 버전 관리
- 6개월마다 갱신

```python
@dataclass
class GoldenTestCase:
    id: str
    category: str   # "accuracy" | "faithfulness" | "robustness"
    input: dict
    expected_output: str
    tolerance: float = 0.8  # 합격 기준 점수
```

퀴즈

Faithfulness 지표가 낮다는 것은 무엇을 의미하는가?

A. 응답 속도가 느리다

B. 정답 데이터와 일치하지 않는다

C. 문서에 근거하지 않는 환각이 많다

D. 모델 크기가 작다

퀴즈

Faithfulness 지표가 낮다는 것은 무엇을 의미하는가?

A. 응답 속도가 느리다

B. 정답 데이터와 일치하지 않는다

C. 문서에 근거하지 않는 환각이 많다

D. 모델 크기가 작다

퀴즈

Faithfulness 지표가 낮다는 것은 무엇을 의미하는가?

A. 응답

치하지 않는다

C. 문서에 근거하지 않는 환각이 많다

C. 문서에 근거

가 작다

Session 2

Prompt · RAG · Tool 성능 개선 전략

성능 저하의 3가지 원인

Prompt Drift

여러 번 수정되며 의도가 흐려진다

Retrieval Drift

문서 업데이트로 검색 결과가 달라진다

Tool Drift

API·스키마 변경으로 호출 실패가 늘어난다

성능 저하의 3가지 원인

Prompt Drift

여러 번 수정되며 의도가 흐려진다

Retrieval Drift

문서 업데이트로 검색 결과가 달라진다

Tool Drift

API·스키마 변경으로 호출 실패가 늘어난다

원인을 모르면 고칠 수 없다. 진단이 개선의 시작이다.

성능 저하 진단 4단계

```
1단계: 증상 확인
   → 어느 지표가 떨어졌는가?

2단계: 계층 격리
   → LLM 자체? 검색? Tool 호출?

3단계: 시점 특정
   → 언제부터? 그 시점에 무엇이 바뀌었나?

4단계: 재현
   → Golden Test로 추가 → 수정 전후 점수 비교
```

성능 저하 진단 4단계

```
1단계: 증상 확인
   → 어느 지표가 떨어졌는가?

2단계: 계층 격리
   → LLM 자체? 검색? Tool 호출?

3단계: 시점 특정
   → 언제부터? 그 시점에 무엇이 바뀌었나?

4단계: 재현
   → Golden Test로 추가 → 수정 전후 점수 비교
```

4단계를 지키면 "감으로 고치는" 함정을 피할 수 있다

Prompt 버전 관리

SemVer 규칙

변경 유형	버전
형식만 수정	Patch (v1.0.1)
동작 개선	Minor (v1.1.0)
목적 변경	Major (v2.0.0)

Prompt 버전 관리

SemVer 규칙

변경 유형	버전
형식만 수정	Patch (v1.0.1)
동작 개선	Minor (v1.1.0)
목적 변경	Major (v2.0.0)

- 코드 저장소에 **파일로 관리**

Prompt 버전 관리

SemVer 규칙

변경 유형	버전
형식만 수정	Patch (v1.0.1)
동작 개선	Minor (v1.1.0)
목적 변경	Major (v2.0.0)

- 코드 저장소에 **파일로 관리**
- CHANGELOG에 변경 이유 기록

Prompt 버전 관리

SemVer 규칙

변경 유형	버전
형식만 수정	Patch (v1.0.1)
동작 개선	Minor (v1.1.0)
목적 변경	Major (v2.0.0)

- 코드 저장소에 **파일로 관리**
- CHANGELOG에 변경 이유 기록
- A/B 테스트로 성능 검증 후 전환

Prompt 버전 관리

SemVer 규칙

변경 유형	버전
형식만 수정	Patch (v1.0.1)
동작 개선	Minor (v1.1.0)
목적 변경	Major (v2.0.0)

- 코드 저장소에 **파일로 관리**
- CHANGELOG에 변경 이유 기록
- A/B 테스트로 성능 검증 후 전환
- 롤백 기능 필수

Retrieval Drift 감지

```
Retrieval Stability Score
= |t2 top-k| ∩ |t1 top-k| / k
```

Retrieval Drift 감지

```
Retrieval Stability Score
= |t2 top-k| ∩ |t1 top-k| / k
```

- t1에서 A, B, C → t2에서 A, D, E → Score = 1/3

Retrieval Drift 감지

```
Retrieval Stability Score
= |t2 top-k| ∩ |t1 top-k| / k
```

- t1에서 A, B, C → t2에서 A, D, E → Score = 1/3
- **임계값 0.7 이하** → 드리프트 경보 발생

Retrieval Drift 감지

```
Retrieval Stability Score
= |t2 top-k| ∩ |t1 top-k| / k
```

- t1에서 A, B, C → t2에서 A, D, E → Score = 1/3
- **임계값 0.7 이하** → 드리프트 경보 발생
- 원인별 대응:
  - 문서 추가/삭제 → 인덱스 변경 로그 추적
  - 임베딩 모델 변경 → 전체 재인덱스 + 성능 비교

Tool 호출 실패 4유형

Type 1 — Wrong Tool

엉뚱한 Tool 선택

→ Tool 설명 개선

Type 2 — Wrong Params

잘못된 파라미터

→ Few-shot 예시 추가

Type 3 — No Validation

파라미터 범위/타입 오류

→ Validation 레이어 추가

Type 4 — Schema Mismatch

API 스키마 변경 미반영

→ 스키마 변경 자동 감지

성능 개선 우선순위

P0 — 즉시 수정

높은 영향 + 높은 빈도

P1 — 다음 스프린트

높은 영향 + 낮은 빈도

P2 — 자동화로 억제

낮은 영향 + 높은 빈도

P3 — 백로그

낮은 영향 + 낮은 빈도

퀴즈

모델 버전 업그레이드 전 반드시 해야 할 것은?

A. Fine-tuning 재수행

B. Golden Test Set 회귀 테스트

C. 임베딩 모델 교체

D. 서버 스케일 업

퀴즈

모델 버전 업그레이드 전 반드시 해야 할 것은?

A. Fine-tuning 재수행

B. Golden Test Set 회귀 테스트

C. 임베딩 모델 교체

D. 서버 스케일 업

퀴즈

모델 버전 업그레이드 전 반드시 해야 할 것은?

A. Fine-

B. Golden Test Set 회귀 테스트

같은 프롬프트도 모델 버전마다 다르게 동작한다

et 회귀 테스트

C. 임비

일 업

Session 3

로그 · 모니터링 · 장애 대응 설계

로그 없이 장애 대응하면

로그 없이 장애 대응하면

- 언제 문제가 생겼는지 모른다

로그 없이 장애 대응하면

- 언제 문제가 생겼는지 모른다
- 영향 범위를 파악할 수 없다

로그 없이 장애 대응하면

- 언제 문제가 생겼는지 모른다
- 영향 범위를 파악할 수 없다
- 재현이 안 되면 수정도 안 된다

로그 없이 장애 대응하면

- 언제 문제가 생겼는지 모른다
- 영향 범위를 파악할 수 없다
- 재현이 안 되면 수정도 안 된다
- "다시 배포하면 되겠지"가 반복된다

로그 없이 장애 대응하면

- 언제 문제가 생겼는지 모른다
- 영향 범위를 파악할 수 없다
- 재현이 안 되면 수정도 안 된다
- "다시 배포하면 되겠지"가 반복된다

> 이커머스 사례: Tool 무한 루프 → 로그 없음 → 영향 범위 파악에 **6시간** 소요 로그 설계 후 동일 장애 → **15분** 해결

Trace 계층 구조

```
Trace (요청 단위, trace_id)
  └── Span (스텝 단위)
        ├── LLM Span  → 모델 호출
        ├── Tool Span → Tool 실행
        └── RAG Span  → 검색 + 청크 반환
```

Trace 계층 구조

```
Trace (요청 단위, trace_id)
  └── Span (스텝 단위)
        ├── LLM Span  → 모델 호출
        ├── Tool Span → Tool 실행
        └── RAG Span  → 검색 + 청크 반환
```

- 하나의 **요청 = 하나의 Trace**

Trace 계층 구조

```
Trace (요청 단위, trace_id)
  └── Span (스텝 단위)
        ├── LLM Span  → 모델 호출
        ├── Tool Span → Tool 실행
        └── RAG Span  → 검색 + 청크 반환
```

- 하나의 **요청 = 하나의 Trace**
- 하나의 **처리 단계 = 하나의 Span**

Trace 계층 구조

```
Trace (요청 단위, trace_id)
  └── Span (스텝 단위)
        ├── LLM Span  → 모델 호출
        ├── Tool Span → Tool 실행
        └── RAG Span  → 검색 + 청크 반환
```

- 하나의 **요청 = 하나의 Trace**
- 하나의 **처리 단계 = 하나의 Span**
- `trace_id` 로 전체 흐름을 추적

Trace 계층 구조

```
Trace (요청 단위, trace_id)
  └── Span (스텝 단위)
        ├── LLM Span  → 모델 호출
        ├── Tool Span → Tool 실행
        └── RAG Span  → 검색 + 청크 반환
```

- 하나의 **요청 = 하나의 Trace**
- 하나의 **처리 단계 = 하나의 Span**
- `trace_id` 로 전체 흐름을 추적
- `parent_span_id` 로 계층 관계 표현

Trace 필수 필드

필드	설명	필드	설명
`trace_id`	요청 전체 ID	`input_tokens`	입력 토큰 수
`span_id`	개별 스텝 ID	`output_tokens`	출력 토큰 수
`parent_span_id`	상위 Span	`model`	사용 모델
`timestamp`	ISO 8601	`status`	success/error
`latency_ms`	처리 시간	`user_id`	사용자 ID (마스킹)

장애 유형 4분류

Type A — LLM 품질

환각, 형식 오류, 부적절한 응답

감지: Faithfulness/Accuracy 모니터링

Type B — Tool 실행

호출 실패, 무한 루프, 잘못된 파라미터

감지: Tool 성공률, 타임아웃

Type C — RAG

검색 결과 없음, 무관한 문서 반환

감지: Retrieval 품질 지표

Type D — 시스템

타임아웃, OOM, 서비스 다운

감지: 인프라 메트릭

5-Why: 근본 원인 분석

```
문제: 잘못된 정책 정보를 반환했다.

Why 1 → 구버전 정책 문서가 검색됐다.
Why 2 → 정책 업데이트 시 인덱스가 갱신되지 않았다.
Why 3 → 문서 업데이트 파이프라인에 인덱스 트리거가 없다.
Why 4 → 초기 설계 시 문서 업데이트 빈도를 과소평가했다.
Why 5 → 정책 담당자가 개발팀과 연결되지 않았다.

근본 원인: 부서 간 프로세스 미정의
```

5-Why: 근본 원인 분석

```
문제: 잘못된 정책 정보를 반환했다.

Why 1 → 구버전 정책 문서가 검색됐다.
Why 2 → 정책 업데이트 시 인덱스가 갱신되지 않았다.
Why 3 → 문서 업데이트 파이프라인에 인덱스 트리거가 없다.
Why 4 → 초기 설계 시 문서 업데이트 빈도를 과소평가했다.
Why 5 → 정책 담당자가 개발팀과 연결되지 않았다.

근본 원인: 부서 간 프로세스 미정의
```

증상이 아닌 근본 원인을 고쳐야 재발이 없다

Guardrail 적용 위치

```
[사용자 입력]
    ↓
[Input Guardrail]    ← PII · 인젝션 · 유해 콘텐츠 감지
    ↓
[Agent 처리]
    ↓
[Tool Call Validation]  ← 파라미터 타입/범위 검증
    ↓
[Output Guardrail]   ← 민감 정보 필터링 · 형식 검증
    ↓
[사용자 응답]
```

Guardrail 적용 위치

```
[사용자 입력]
    ↓
[Input Guardrail]    ← PII · 인젝션 · 유해 콘텐츠 감지
    ↓
[Agent 처리]
    ↓
[Tool Call Validation]  ← 파라미터 타입/범위 검증
    ↓
[Output Guardrail]   ← 민감 정보 필터링 · 형식 검증
    ↓
[사용자 응답]
```

입력·출력 양쪽에서 모두 검사해야 한다

알림 임계값 설계

임계값	채널	대응 시간
오류율 > 5% (5분)	PagerDuty	즉시 (On-call)
Faithfulness < 0.6	Slack	2시간 내
평균 응답 > 10s	Slack	업무 시간 내
토큰 비용 > 예산 80%	이메일	다음날

알림 임계값 설계

임계값	채널	대응 시간
오류율 > 5% (5분)	PagerDuty	즉시 (On-call)
Faithfulness < 0.6	Slack	2시간 내
평균 응답 > 10s	Slack	업무 시간 내
토큰 비용 > 예산 80%	이메일	다음날

알림이 너무 많으면 알림 피로(Alert Fatigue)가 생긴다

퀴즈

대규모 운영 환경에서 권장하는 로깅 전략은?

A. 전수 로깅

B. 오류만 로깅

C. 오류 전수 + 정상 10% 샘플링

D. 메트릭만 수집

퀴즈

대규모 운영 환경에서 권장하는 로깅 전략은?

A. 전수 로깅

B. 오류만 로깅

C. 오류 전수 + 정상 10% 샘플링

D. 메트릭만 수집

퀴즈

대규모 운영 환경에서 권장하는 로깅 전략은?

A. 로깅

C. 오류 전수 + 수집

C. 오류 전수 + 정상 10% 샘플링

비용과 디버깅 능력의 균형

Session 4

확장 가능한 서비스 아키텍처

환경 분리 없이 성장하면

환경 분리 없이 성장하면

- 개발 중 실험이 운영 데이터에 영향을 준다

환경 분리 없이 성장하면

- 개발 중 실험이 운영 데이터에 영향을 준다
- 긴급 패치가 Prod에 바로 배포된다

환경 분리 없이 성장하면

- 개발 중 실험이 운영 데이터에 영향을 준다
- 긴급 패치가 Prod에 바로 배포된다
- 환경별 설정이 코드에 하드코딩된다

환경 분리 없이 성장하면

- 개발 중 실험이 운영 데이터에 영향을 준다
- 긴급 패치가 Prod에 바로 배포된다
- 환경별 설정이 코드에 하드코딩된다
- 환경 분리 리팩터링에 **수 주**가 소요된다

환경 분리 없이 성장하면

- 개발 중 실험이 운영 데이터에 영향을 준다
- 긴급 패치가 Prod에 바로 배포된다
- 환경별 설정이 코드에 하드코딩된다
- 환경 분리 리팩터링에 **수 주**가 소요된다

> 스타트업 사례: B2B 고객 20곳 붙은 후 환경 분리에 6주 소요 → 기능 개발 전면 중단

Dev · Staging · Prod 역할

환경	목적	데이터	모델
Dev	개발·실험	합성/샘플	gpt-4o-mini
Staging	검증·QA	익명화 복사본	운영과 동일
Prod	실제 서비스	실제 운영	운영 모델

Dev · Staging · Prod 역할

환경	목적	데이터	모델
Dev	개발·실험	합성/샘플	gpt-4o-mini
Staging	검증·QA	익명화 복사본	운영과 동일
Prod	실제 서비스	실제 운영	운영 모델

- 환경 변수로 모델명·API 키·인덱스를 분리
- `config/dev.yaml`, `config/prod.yaml` 로 관리
- 코드에 환경명 하드코딩 **절대 금지**

Scaling 전략 비교

수직 Scaling (Scale Up)

수평 Scaling (Scale Out)

Scaling 전략 비교

수직 Scaling (Scale Up)

- 서버 사양(CPU/메모리) 증가

수평 Scaling (Scale Out)

Scaling 전략 비교

수직 Scaling (Scale Up)

- 서버 사양(CPU/메모리) 증가
- 한계가 명확함

수평 Scaling (Scale Out)

Scaling 전략 비교

수직 Scaling (Scale Up)

- 서버 사양(CPU/메모리) 증가
- 한계가 명확함
- 비용이 급격히 증가

수평 Scaling (Scale Out)

Scaling 전략 비교

수직 Scaling (Scale Up)

- 서버 사양(CPU/메모리) 증가
- 한계가 명확함
- 비용이 급격히 증가
- 단기 임시방편

수평 Scaling (Scale Out)

Scaling 전략 비교

수직 Scaling (Scale Up)

- 서버 사양(CPU/메모리) 증가
- 한계가 명확함
- 비용이 급격히 증가
- 단기 임시방편

수평 Scaling (Scale Out)

- 동일 인스턴스 여러 개 실행

Scaling 전략 비교

수직 Scaling (Scale Up)

- 서버 사양(CPU/메모리) 증가
- 한계가 명확함
- 비용이 급격히 증가
- 단기 임시방편

수평 Scaling (Scale Out)

- 동일 인스턴스 여러 개 실행
- Stateless 설계가 전제

Scaling 전략 비교

수직 Scaling (Scale Up)

- 서버 사양(CPU/메모리) 증가
- 한계가 명확함
- 비용이 급격히 증가
- 단기 임시방편

수평 Scaling (Scale Out)

- 동일 인스턴스 여러 개 실행
- Stateless 설계가 전제
- 세션 상태 → Redis/DB에 저장

Scaling 전략 비교

수직 Scaling (Scale Up)

- 서버 사양(CPU/메모리) 증가
- 한계가 명확함
- 비용이 급격히 증가
- 단기 임시방편

수평 Scaling (Scale Out)

- 동일 인스턴스 여러 개 실행
- Stateless 설계가 전제
- 세션 상태 → Redis/DB에 저장
- Agent 서비스 권장 방식

Stateless 설계의 핵심

상태를 서버 메모리에 저장

→ 서버 A에서 시작한 요청이

→ 서버 B로 가면 상태 유실

→ Scale Out 불가

상태를 외부 저장소에 저장

→ 모든 서버가 Redis/DB에서 읽음

→ 어느 서버도 요청 처리 가능

→ Scale Out 자유롭게 가능

Multi-Agent 3가지 패턴

```
패턴 1: Orchestrator-Worker
  Orchestrator → Task 분해 → Worker에게 위임
  적합: 복잡한 장문 Task

패턴 2: Pipeline (순차)
  Intake → Classify → Retrieve → Generate → Validate
  적합: 명확한 처리 흐름

패턴 3: Parallel Fan-out
  동일 입력 → 여러 Agent 병렬 처리 → 결과 합산
  적합: 다각도 분석, 교차 검증
```

Multi-Agent 전환 기준

조건	단일 Agent	Multi-Agent
Task 복잡도	단순 (≤3 스텝)	복잡 (5+ 스텝)
컨텍스트 길이	짧음	길거나 누적됨
도메인 전문성	범용	여러 전문 영역
오류 격리 필요	낮음	높음

Multi-Agent 전환 기준

조건	단일 Agent	Multi-Agent
Task 복잡도	단순 (≤3 스텝)	복잡 (5+ 스텝)
컨텍스트 길이	짧음	길거나 누적됨
도메인 전문성	범용	여러 전문 영역
오류 격리 필요	낮음	높음

> "지금 당장 필요한가?" — 항상 이 질문을 먼저 한다

모델 계층화 전략

단순 분류/필터

gpt-4o-mini

저비용 · 빠름

RAG 응답 생성

gpt-4o

균형

복잡한 추론

o3

고비용 · 최소화

모델 계층화 전략

단순 분류/필터	RAG 응답 생성	복잡한 추론
gpt-4o-mini	gpt-4o	o3
저비용 · 빠름	균형	고비용 · 최소화

모든 호출에 같은 모델을 쓰면 비용이 10배 이상 늘어난다

비용 최적화 3가지 전략

모델 계층화

Task별 최적 모델 사용

효과: ★★★★★

Semantic Cache

유사 쿼리 캐시 히트

효과: ★★★★☆

컨텍스트 압축

히스토리 요약으로 토큰 절감

효과: ★★★☆☆

카나리 배포 파이프라인

```
코드 커밋
  ↓
CI (단위 테스트 + Golden Test Set)
  ↓
Dev 배포 (자동)
  ↓
Staging 배포 (자동) + 통합 테스트
  ↓
Prod 카나리 5% → 50% → 100%
  ↓
이상 감지 시 자동 롤백
```

카나리 배포 파이프라인

```
코드 커밋
  ↓
CI (단위 테스트 + Golden Test Set)
  ↓
Dev 배포 (자동)
  ↓
Staging 배포 (자동) + 통합 테스트
  ↓
Prod 카나리 5% → 50% → 100%
  ↓
이상 감지 시 자동 롤백
```

- 5%에서 **1시간 관찰**
- 오류율·응답 시간 기준 통과 후 확대
- 문제 발생 → 즉시 이전 버전 롤백

퀴즈

Agent 서비스가 수평 Scaling을 위해 반드시 지켜야 하는 원칙은?

A. 모든 상태를 서버 메모리에 저장

B. Stateless 설계 + 외부 저장소 사용

C. 단일 서버에서 스펙을 높인다

D. 모든 요청을 동기식으로 처리

퀴즈

Agent 서비스가 수평 Scaling을 위해 반드시 지켜야 하는 원칙은?

A. 모든 상태를 서버 메모리에 저장

B. Stateless 설계 + 외부 저장소 사용

C. 단일 서버에서 스펙을 높인다

D. 모든 요청을 동기식으로 처리

퀴즈

Agent 서비스가 수평 Scaling을 위해 반드시 지켜야 하는 원칙은?

A. 모든 상태를 외부 저장소 사용

B. Stateless 설계 + 외부 저장소

세션 상태는 Redis/DB에 저장해야 Scale Out이 가능

C. 단일 서버에 기식으로 처리

Day 4 정리

Day 4 핵심 요약

Day 4 핵심 요약

- **평가 3축**: Accuracy · Faithfulness · Robustness를 독립 측정

Day 4 핵심 요약

- **평가 3축**: Accuracy · Faithfulness · Robustness를 독립 측정
- **LM-as-a-Judge**: 편향 제거(순서 무작위화, 모델 분리) 후 사용

Day 4 핵심 요약

- **평가 3축**: Accuracy · Faithfulness · Robustness를 독립 측정
- **LM-as-a-Judge**: 편향 제거(순서 무작위화, 모델 분리) 후 사용
- **Golden Test Set**: 격리 저장, 버전 관리, 6개월마다 갱신

Day 4 핵심 요약

- **평가 3축**: Accuracy · Faithfulness · Robustness를 독립 측정
- **LM-as-a-Judge**: 편향 제거(순서 무작위화, 모델 분리) 후 사용
- **Golden Test Set**: 격리 저장, 버전 관리, 6개월마다 갱신
- **진단 4단계**: 증상 → 계층 격리 → 시점 특정 → 재현

Day 4 핵심 요약

- **평가 3축**: Accuracy · Faithfulness · Robustness를 독립 측정
- **LM-as-a-Judge**: 편향 제거(순서 무작위화, 모델 분리) 후 사용
- **Golden Test Set**: 격리 저장, 버전 관리, 6개월마다 갱신
- **진단 4단계**: 증상 → 계층 격리 → 시점 특정 → 재현
- **Trace 로그**: Trace/Span 계층 구조로 전체 흐름 추적

Day 4 핵심 요약

- **평가 3축**: Accuracy · Faithfulness · Robustness를 독립 측정
- **LM-as-a-Judge**: 편향 제거(순서 무작위화, 모델 분리) 후 사용
- **Golden Test Set**: 격리 저장, 버전 관리, 6개월마다 갱신
- **진단 4단계**: 증상 → 계층 격리 → 시점 특정 → 재현
- **Trace 로그**: Trace/Span 계층 구조로 전체 흐름 추적
- **Guardrail**: 입력·Tool·출력 세 지점에서 검증

Day 4 핵심 요약

- **평가 3축**: Accuracy · Faithfulness · Robustness를 독립 측정
- **LM-as-a-Judge**: 편향 제거(순서 무작위화, 모델 분리) 후 사용
- **Golden Test Set**: 격리 저장, 버전 관리, 6개월마다 갱신
- **진단 4단계**: 증상 → 계층 격리 → 시점 특정 → 재현
- **Trace 로그**: Trace/Span 계층 구조로 전체 흐름 추적
- **Guardrail**: 입력·Tool·출력 세 지점에서 검증
- **환경 분리**: Dev/Staging/Prod 데이터와 모델을 분리

Day 4 핵심 요약

- **평가 3축**: Accuracy · Faithfulness · Robustness를 독립 측정
- **LM-as-a-Judge**: 편향 제거(순서 무작위화, 모델 분리) 후 사용
- **Golden Test Set**: 격리 저장, 버전 관리, 6개월마다 갱신
- **진단 4단계**: 증상 → 계층 격리 → 시점 특정 → 재현
- **Trace 로그**: Trace/Span 계층 구조로 전체 흐름 추적
- **Guardrail**: 입력·Tool·출력 세 지점에서 검증
- **환경 분리**: Dev/Staging/Prod 데이터와 모델을 분리
- **모델 계층화**: Task 복잡도별 최적 모델로 비용 최적화

Day 4 실습 안내

Lab 1 — Golden Test Evaluation

- Golden Test Set 20건 설계
- LM-as-a-Judge 평가 파이프라인
- Faithfulness 자동 측정
- 결과 대시보드 출력

Lab 2 — 운영 아키텍처 설계

- 환경 분리 설계서 작성
- Scaling 전략 결정
- 월간 비용 추정 계산
- 카나리 배포 계획

수고하셨습니다

Day 4 — 평가 · 운영 · 확장 아키텍처 전략

> 만든 것을 측정하고, 운영하고, 확장하는 것이 진짜 엔지니어링이다.